

1. Objectives:

1. To understand the basic concept of a Doubly Linked List.
2. To learn the operations of insertion at the beginning, at the end, deletion of the first node, last node, and traversal.
3. To analyze the advantages, working mechanism, and limitations of Doubly Linked Lists compared to linear data structures like arrays.

2. Introduction:

- **Doubly Linked List:** Doubly Linked List is an advanced type of linked list where each node contains a value and two pointers: one pointing to the next node and another pointing to the previous node. The first node is referenced by a pointer called start or head, and the last node points to NULL in its next pointer, while its previous pointer connects to the previous node. In this structure, each node consists of three parts: data, a pointer to the next node, and a pointer to the previous node. This allows traversal in both forward and backward directions, making it more flexible than a singly linked list.

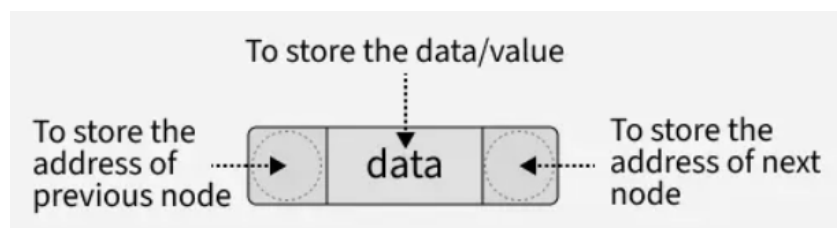


Fig 6.1: Singly Linked List

To insert a new node at the beginning, a node is created and its next pointer is set to the current head, while the current head's previous pointer is updated to the new node, and then the head is moved to the new node. For insertion at the end, the list is traversed until the last node, and the new node is attached by updating the last node's next pointer and the new node's previous pointer.

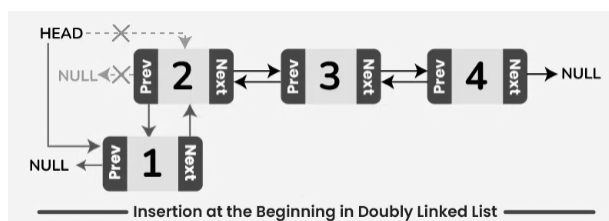


Fig 6.2: Insertion at start

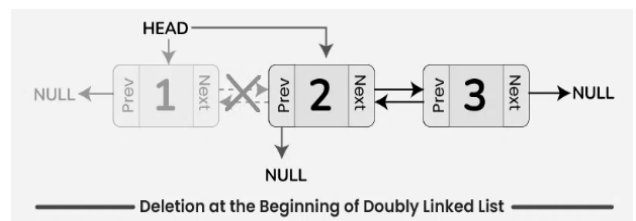


Fig 6.3: Deletion at start

To delete a node, pointer adjustment is required carefully in both directions. For deleting the first node, the head is moved to the next node, and its previous pointer is set to NULL. For deleting the last node, the list is traversed to the second last node, and its next pointer is set to NULL, removing the last node.

- **Tortoise and Hare Algorithm:** The algorithm is to start two pointers, slow and fast from head of linked list. We move slowly one node at a time and fast two nodes at a time. If there is a loop, then they will meet at some point. After each iteration where the **slow** pointer moves one step forward and the **fast** pointer moves two steps forward, the distance between the two pointers increases. Initially, if the **slow** pointer is at a distance k from a certain point in the cycle, then after one iteration, the distance between the **slow** and **fast** pointers becomes $k+1$. After two

iterations, this distance becomes $k+2$ and so on. As they continue to move within the cycle, this distance will eventually equal the cycle length n . At this point, since the distance wraps around the cycle and both pointers are moving within the same cycle, they will meet.

3. Procedure

Experiment 1: Doubly Linked List

Input:

- k – Integer value to be inserted into the linked list
- $start$ – Pointer to the first node of the list

Output:

- Printed values of all nodes in the linked list

Algorithm:

InsertStart($start, k$):

1. $n \leftarrow \text{new Node}(k)$
2. **if** $start == \text{NULL}$ **then**
3. $start \leftarrow n$
4. **else**
5. $n \rightarrow \text{next} \leftarrow start$
6. $start \rightarrow \text{prev} \leftarrow n$
7. $start \leftarrow n$

InsertEnd($start, k$):

1. $n \leftarrow \text{new Node}(k)$
2. **if** $start == \text{NULL}$ **then**
3. $start \leftarrow n$
4. **else**
5. $p \leftarrow start$
6. **while** $p \rightarrow \text{next} \neq \text{NULL}$ **do**
7. $p \leftarrow p \rightarrow \text{next}$
8. $p \rightarrow \text{next} \leftarrow n$
9. $n \rightarrow \text{prev} \leftarrow p$

DeleteFirst($start$):

1. $start \leftarrow start \rightarrow \text{next}$
2. $start \rightarrow \text{prev} \leftarrow \text{NULL}$

DeleteLast($start$):

1. $j \leftarrow start$
2. **while** $j \rightarrow \text{next} \rightarrow \text{next} \neq \text{NULL}$ **do**
3. $j \leftarrow j \rightarrow \text{next}$
4. $j \rightarrow \text{next} \leftarrow \text{NULL}$

Traverse($start$):

1. $p \leftarrow start$
2. **while** $p \rightarrow \text{next} \neq \text{NULL}$ **do**
3. **print** $p \rightarrow \text{val}$
4. $p \leftarrow p \rightarrow \text{next}$

```

5. print p -> val
6. Print newline
7. while p != Null do
8.     print p -> val
9.     p ← p->next
10. print newline

```

Experiment 2: Hare and Tortoise Algorithm

Input:

- k – Integer value to be inserted
- start – Pointer to first node of the list

Output:

- Printed linked list
- Middle element of the first node of the linked list

Algorithm:

InsertEnd(start, k):

```

10. n ← new Node(k)
11. if start == NULL then
12.     start ← n
13. else
14.     p ← start
15.     while p->next != NULL do
16.         p ← p->next
17.     p->next ← n

```

InsertBeginning(start, k):

```

1. n ← new Node(k)
2. if start == NULL then
3.     start ← n
4. else
5.     n->next ← start
6.     start ← n

```

FindMiddle(start):

```

1. ash ← start
2. sif ← start
3. while ash != NULL and ash->next != NULL do
4.     sif ← sif->next
5.     ash ← ash->next->next
6. print "Middle element is: ", sif-> val

```

Traverse(start):

```

1. p ← start
2. while p != NULL do
3.     print p->val
4.     p ← p->next
5. print newline

```

4. Required Hardware and Software:

- **Hardware:** AMD RYZEN 7, RAM 8 GB

- **Software:** Visual Studio Code 1.109

5. Experiment Implementation:

Experiment 1: Doubly Linked List

```
#include <bits/stdc++.h>
using namespace std;
class Node{
public:
    int val;
    Node *next;
    Node *prev;
    Node(int k){
        val = k;
        next = nullptr;
        prev = nullptr;
    }
};

class List{
public:
    Node *start = nullptr;

    void traverse(){
        Node *p = start;
        while (p->next != nullptr){
            cout << p->val << "\t";
            p = p->next;
        }
        cout << p->val << "\t";
        cout << endl;

        while (p != nullptr){
            cout << p->val << "\t";
            p = p->next;
        }
        cout << endl;
    }

    void insFirst(int k){
        Node *n = new Node(k);
        if (start == nullptr){
            start = n;
        }
        else{
            n->next = start;
            start->prev = n;
            start = n;
        }
    }

    void insLast(int k){
        Node *n = new Node(k);
        if (start == nullptr){
            start = n;
        }
        else{
            Node *p = start;
            while (p->next != nullptr)
            {
                p = p->next;
            }
            p->next = n;
            n->prev = p;
        }
    }

    void deleteFirst(){
        start = start->next;
        start->prev = nullptr;
    }
}
```

```

void deleteFirst(){
    start = start->next;
    start->prev = nullptr;
}

void deleteLast(){
    Node *j = start;
    while (j->next->next != nullptr){
        j = j->next;
    }
    j->next = nullptr;
}

};

int main(){
    List t;
    t.insLast(5);
    t.insLast(10);
    t.insLast(3);
    t.insLast(6);
    t.traverse();
    t.insFirst(9);
    t.traverse();
    t.deleteFirst();
    t.traverse();
    t.deleteLast();
    t.traverse();
    return 0;
}

```

Table 5.1: Experiment 1 output

Input	Expected Output				Actual Output			
	5	10	3	6	5	10	3	6
	6				6			
	9	5	10	3	9	5	10	3
	6				6			
	5	10	3	6	5	10	3	6
	6				6			
	5	10	3		5	10	3	
	3				3			

Experiment 2: Singly Linked List Insert

```

#include <bits/stdc++.h>
using namespace std;
class Node{
public:
    int val;
    Node *next;
    Node(int k){
        val = k;
        next = NULL;
    }
};

class List{
public:
    Node *start = NULL;
    void ins(int k){
        Node *n = new Node(k);
        if (start == NULL){
            start = n;
        }
        else{
            Node *p = start;
            while (p->next != NULL){
                p = p->next;
            }
            p->next = n;
        }
    }

    void insLast(int k){

```

```

        Node *n = new Node(k);
        if (start == NULL){
            start = n;
        }
        else{
            n->next = start;
            start = n;
        }
    }

    void middleElement(){
        Node *ash = start;
        Node *sif = start;
        while (ash != NULL && ash->next != NULL){
            sif = sif->next;
            ash = ash->next->next;
        }
        cout << "Middle element is: " << sif->val << endl;
    }

    void traverse(){
        Node *p = start;
        while (p != NULL){
            cout << p->val << "\t";
            p = p->next;
        }
        cout << endl;
    }
};

int main(){
    List t;
    t.ins(5);
    t.ins(10);
    t.ins(3);
    t.ins(6);
    t.insLast(9);
    t.traverse();
    t.middleElement();
    return 0;
}

```

Table 5.2: Experiment 2 output

Input	Expected Output	Actual Output
	9 5 10 3 6 Middle element is: 10	9 5 10 3 6 Middle element is: 10

6. Discussion & Conclusion:

In this lab we successfully implemented different operations of Doubly linked lists and Tortoise and Hare algorithm, including insertion at the beginning and end, deletion of nodes, and traversal of elements. Pointer manipulation is the core concept behind linked lists to maintain correct connections between nodes. The doubly linked list provided the advantage of bidirectional traversal. Additionally, the Tortoise and Hare algorithm was used to find the middle element efficiently without using extra space, demonstrating an optimized approach compared to traditional methods. We faced difficulty in edge cases, such as deletion when the list contains a small number of nodes and ensuring correct pointer updates. Overall, this experiment enhanced our understanding of dynamic data structures, pointer operations, and efficient algorithm design using linked lists.

7. Reference:

1. <https://www.geeksforgeeks.org/dsa/doubly-linked-list/> accessed at 7.25 AM on 12th April 2026.
2. <https://www.geeksforgeeks.org/dsa/insert-a-node-at-frontbeginning-of-doubly-linked-list/> accessed at 11.00 AM on 12th April 2026